A Mechanized Library of Finite Game Theory: Definitional Supplement

Elazar Gershuni
elazarg@gmail.com

May 2026

Abstract

This document is a companion to the main chapter *A Mechanized Library of Finite Game Theory*. The chapter states results and explains design choices; this supplement gives definitions. Its primary purpose is faithfulness: the reader should leave convinced that what the Lean library calls *Nash equilibrium*, *correlated equilibrium*, *Kuhn realization*, *Vickrey auction*, and the rest, is in each case the textbook concept of the same name. We anchor each major notion with a full declaration shape and an explicit textbook correspondence, and group structurally similar variants in bunched entries so the reader can confirm that the variants are routine specializations of the anchor.

Contents

1 How To Read This Document

1.1 Conventions

Lean boxes. Each definition appears as a shaded *Lean box* containing the declaration shape used in the code. Universe-polymorphism boilerplate, routine implicit binders, and `Fintype`/`DecidableEq` typeclass arguments are suppressed when they would obscure the content; the file:line reference points to the verbatim source. Defined names link to their declaration via `KernelGame`-style hyperlinks; the linked anchor is the real definition site of the name (here, §3.2). Identifiers that are not project concepts — `PMF`, `Function.update`, `Finset`, `Fintype` — are typeset `monospace` but not hyperlinked.

Anchors versus bunched entries. A faithful mechanization of a textbook tradition cannot show every definition in full and remain readable. We therefore distinguish two formats.

- An *anchor* entry shows the full Lean declaration, gives the corresponding mathematical content explicitly, cites a textbook page where the same content appears, and (where applicable) names the worked example in §9 in which the predicate is unfolded by hand on a small game. Anchor entries do the convincing work of the supplement.
- A *bunched* entry groups several structurally similar definitions in one Lean box, giving the mathematical content of one and a one-line note on each variant. The reader trusts that the variants are routine specializations of the anchor; the bunched format makes the relationships visible.

Faithfulness gaps. Where the Lean definition diverges from the textbook (and several do, deliberately), the gap is named, in one of three buckets:

- *Strict generalization.* The Lean form is more general than the textbook (e.g. `IsNashFor` takes any preference relation; the textbook fixes EU). The reduction back to the textbook case is shown.
- *Genuine restriction.* The Lean form is finite/discrete where the textbook is general (e.g. Bayesian games on continuous type spaces). The restriction is stated.
- *Reformulation.* Same content, different shape (e.g. distributional EFG semantics rather than deterministic-with-lift). The reformulation is justified once.

1.2 Lean syntax mini-glossary

A reader who knows finite game theory but not Lean 4 will find a handful of constructs in the boxes below that look like ordinary mathematical objects but mean something more precise. We gloss the ones that recur:

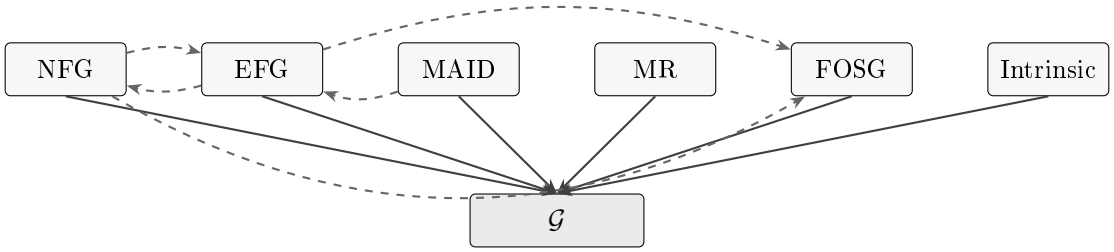
- $\forall i, S\,i$, often written `(i : Îź) âĘŠ S i`, is the *dependent function type*: for each i in the index type $\mathcal{I}$, an element of $S\,i$. When S is a strategy family, an element of this type is exactly a strategy profile.
- `structure` declares a record (named tuple). `KernelGame Îź` below is a record with four fields; we write $(\text{Strat}, \mathcal{O}, u, K) : \mathcal{G}_{\mathcal{I}}$ in math.

- `[Fintype Î́ś]` is a typeclass argument: Lean's inference looks up a finite-enumeration witness from context. Reading "the theorem requires `[Fintype $\Strat _i$]`" is reading "the theorem requires that each strategy space be finite."
- `noncomputable` marks a definition that compiles into a proof object but does not reduce by computation — typically because it invokes `Classical.choice` or real-number arithmetic. The mathematical content is unchanged.
- `open Classical in` turns on the law of excluded middle for the duration of one declaration. This is benign and is flagged at every use.
- `abbrev` is a definition that unfolds eagerly (in elaboration and in `simp`). We use it for `Profile` and `Payoff`.
- `inductive` declares an inductive type. `GameTree S O` in §5.2 has three constructors (terminal, chance, decision), one of which is recursive in the same type.
- `Prop` is the universe of propositions; `Type` is the universe of data. Solution concepts are `Prop`-valued.

The conventions for $\gg=$ on PMF($\cdot$), the meaning of δ_x, and the use of pmfPi for the independent product follow §2.

1.3 Convergence diagram

The library has a single semantic core — `KernelGame` (§3.2) — into which every syntactic language compiles, and a graph of *bridges* between languages that commute through it. Every solution concept in §4 is defined once on $\mathcal{G}$ (or on its utility-free shadow `GameForm`) and inherited by every language by composition with $\rightsquigarrow$.



Solid arrows are compilations $\rightsquigarrow$; dashed arrows are bridges (§6). The *Repeated* and *Stochastic* languages are restrictions of MR and are not shown separately. Augmented EFG is a thin wrapper over EFG; it is not a separate language.

1.4 Code organization

Folder	Contents
`Math/`	Discrete-probability infrastructure that mathlib does not already provide: `expect`, `Math.PMFProduct.pmfPi`, finsupp/finset update lemmas, optimization primitives, and the standalone `Math.Schauder.schauder_fixed_point` theorem.
`Core/`	`GameForm`, `KernelGame`, the `ProtocolMap` morphism vocabulary, simulations, isomorphisms, utility-invariance results.
`Concepts/`	Solution concepts: Nash, dominance, best response, correlated / coarse correlated, Pareto, security, regret, approximate Nash, potential, zero-sum, team, symmetric, rationalizability, common knowledge, EU/preference properties.
`Theorems/`	Major theorems: `mixed_nash_exists`, `correlatedEq_exists`, `von_neumann_minimax`, `zermelo`, `oneShotDeviation_iff_spe`, the `ObsModelCore` Kuhn theorems.
`Languages/`	Six syntactic languages (NFG, EFG, MAID, MultiRound, FOSG, Intrinsic), each with its own `Syntax`, `Compile`, and worked-example files; bridges in `Languages/Bridges/`.
`Mechanism/`	Bayesian games, mechanism design, the revelation principle, social-choice fragments.
`Auctions/`	Vickrey, FirstPrice, AllPay, VCG.

2 Probability infrastructure

The library uses mathlib's discrete probability monad `PMF` as its only distribution type. This section documents the few non-mathlib additions in `Math/` that the rest of the library depends on.

2.1 Expected value: `Math.Probability.expect`

```
namespace Math.Probability
noncomputable def expect {Ω : Type*} (d : PMF Ω) (f : Ω → ℝ) : ℝ :=
  ∑' ω, (d ω).toReal * f ω
theorem expect_eq_sum {Ω : Type*} [Fintype Ω] (d : PMF Ω) (f : Ω → ℝ) :
  expect d f = ∑ ω : Ω, (d ω).toReal * f ω
@[simp] theorem expect_pure {Ω : Type*} (f : Ω → ℝ) (ω : Ω) :
  expect (PMF.pure ω) f = f ω
```

Mathematical content. For a PMF(Ω) given by mass function d and a real-valued function $f : \Omega \to \mathbb{R}$,

$$\mathrm{expect}(d, f) \;=\; \sum_{\omega \in \Omega}^{\mathsf{tsum}} d(\omega) \cdot f(\omega), \tag{1}$$

where the sum is mathlib's `tsum` (an unordered series with non-negative-real summands; here the summands are real, but $d \geq 0$ and the unit-mass condition $\sum d = 1$ make absolute convergence automatic for any bounded f, and the convention tsum $= 0$ on non-summable inputs is harmless because the expected-utility theorems call `expect` under boundedness or finite-support hypotheses). When Ω is finite, `expect_eq_sum` reduces this to an ordinary finite sum.

Textbook correspondence. This is the standard expected-value functional of the discrete case (e.g. Maschler–Solan–Zamir [10] §5.4 or Osborne–Rubinstein [16] §1.2). The `toReal` coercion is the only Lean-specific bookkeeping: `PMF`'s mass function lands in $\overline{\mathbb{R}}_{\geq 0}$ (extended non-negative reals) for technical mathlib reasons, and `toReal` brings it back to $\mathbb{R}$.

Faithfulness. Reformulation only. The textbook expectation $\mathbb{E}_\mu[f]$ is exactly (1); the `tsum` appears because mathlib's PMF() is countably-supported in general (the finite-support case is recovered automatically by `expect_eq_sum`, and bounded-f identities such as `expect_bind_of_bounded`, `expect_pushforward_of_bounded`, `expect_pushforward_of_bounded_on_source`, and `expect_mono_of_pointwise_bounded` carry the algebra into the genuinely countable case).

2.2 Indexed product of PMFs: `Math.PMFProduct.pmfPi`

```
namespace Math.PMFProduct
noncomputable def pmfPi {ι :  Type} [Fintype ι] {α :  ι → Type}
   (σ :  ∀ i, PMF (α i)) :  PMF (∀ i, α i)
```

Mathematical content. For a family $\{\sigma_i \in \mathrm{PMF}(\alpha_i)\}_{i \in \mathcal{I}}$ indexed by a finite type $\mathcal{I}$ — the per-coordinate types α_i are otherwise arbitrary and may be countably infinite — $\mathrm{pmfPi}(\sigma)$ is the independent product distribution on $\prod_i \alpha_i$,

$$\mathrm{pmfPi}(\sigma)(x) \;=\; \prod_{i \in \mathcal{I}} \sigma_i(x_i).$$

This is the canonical mixed-strategy joint distribution from independent per-player mixings.

Textbook correspondence. Standard: Osborne–Rubinstein §3.1 use it implicitly when defining mixed-strategy equilibrium expected utility; we make it explicit because the mixed extension of a kernel game (§3.4) needs it as a named operator.

2.3 Other Math/ contents

`Math/Probability.lean` additionally contains the `Kernel`-as-function abbreviation $\mathrm{Kernel}\, A\, B = A \to \mathrm{PMF}(B)$, the `Kernel.pushforward` operator (post-composition with a function, used to define `correlatedOutcome`), `expect_bind` (linearity of `expect` along $\gg\!=$), and `expect_const` (expectation of a constant function). `Math/PMFProduct.lean` adds the projection laws of `pmfPi`, in particular that $\mathrm{pmfPi}(\sigma) \gg\!= \pi_i = \sigma_i$ where π_i is the i-th coordinate projection. These are routine and we do not unfold them. `Math/SchauderFixedPoint.lean` proves Schauder's fixed-point theorem for nonempty compact convex subsets of real normed spaces. It is not part of the finite-dimensional mixed-Nash proof, but it provides a reusable analytic fixed-point principle for compact-convex equilibrium arguments.

3 The semantic core

This section anchors the four objects on which the rest of the library is built: `GameForm`, `KernelGame`, the expected-utility functional `KernelGame.eu`, and the `mixedExtension` construction.

3.1 Game form: `GameForm`

Anchor: `GameForm`.

```
structure GameForm (ι :  Type) where
   Strategy :  ι → Type
   Outcome :  Type
   outcomeKernel :  Kernel (∀ i, Strategy i) Outcome
abbrev Profile (F : GameForm ι) := ∀ i, F.Strategy i
noncomputable def outcomeDist (F : GameForm ι) (σ :  F.Profile) :  PMF F.Outcome :=
   F.outcomeKernel σ
noncomputable def correlatedOutcome (F : GameForm ι) (μ :  PMF F.Profile) :  PMF F.Outcome :=
   Kernel.pushforward F.outcomeKernel μ
```

Mathematical content. A *game form* [14] over a player type $\mathcal{I}$ is the data

$$\mathcal{F} \;=\; \big(\mathsf{S} : \mathcal{I} \to \mathsf{Type},\ \mathcal{O} : \mathsf{Type},\ K : \textstyle\prod_i \mathsf{S}_i \to \mathrm{PMF}(\mathcal{O})\big).$$

A *strategy profile* is an element of $\prod_i \mathsf{S}_i$; a *correlated profile distribution* is an element of $\mathrm{PMF}(\prod_i \mathsf{S}_i)$. The kernel K assigns to each profile a distribution over outcomes; the correlated outcome distribution is the pushforward of the profile distribution along K.

Textbook correspondence. The protocol-only fragment of a game (Myerson §3.1; Maschler–Solan–Zamir §1.2 call it the *game form* explicitly). Decomposing a game into protocol-only data and a separately attached utility is standard mechanism-design practice; the Lean record makes the decomposition structural (`utility` is a field of `KernelGame`, not of `GameForm`).

Faithfulness. Reformulation. The textbook game form is usually presented as the strategy spaces and a deterministic outcome function or a distributional version embedded in a larger structure; we expose the distributional version directly because chance moves and mixed strategies both produce $\mathrm{PMF}(\mathcal{O})$, and the cost of an indirection through a deterministic outcome function plus an external lift is exactly the indirection we wish to avoid (see chapter §3).

Variants. `Core/GameForm.lean` also defines: `deviateProfile` (unilateral coordinate update via `Function.update`); `deviateOutcome`, `deviateDistribution`, `deviateDistributionFn` (push profile or distribution through a deviation, by constant strategy or by a function of the recommendation); `constDeviateDistribution` and its Fn variant; the bridge `withUtility` adjoining a utility (and the round-trip lemmas with `toGameForm`); the functorial action `map` (post-composing a function on outcomes); the *independent product* of two game forms; the `ProtocolMap` morphism record (§3.6).

3.2 Kernel game: `KernelGame`

Anchor: `KernelGame`.

```
abbrev Payoff (ι :  Type) :  Type := ι → ℝ
structure KernelGame (ι :  Type) where
   Strategy :  ι → Type
   Outcome :  Type
   utility :  Outcome → Payoff ι
   outcomeKernel :  Kernel (∀ i, Strategy i) Outcome
abbrev Profile (G : KernelGame ι) := ∀ i, G.Strategy i
def toGameForm (G : KernelGame ι) :  GameForm ι where
   Strategy := G.Strategy
   Outcome := G.Outcome
```

```
    outcomeKernel := G.outcomeKernel
```

Mathematical content. A *kernel game* is a game form together with a utility $u : \mathcal{O} \to \mathcal{I} \to \mathbb{R}$:

$$\mathcal{G} \;=\; \big(\mathsf{S},\ \mathcal{O},\ u,\ K\big), \qquad u(\omega) \in \mathbb{R}^{\mathcal{I}} \text{ for each } \omega.$$

The forgetful map toGameForm takes a kernel game to its game form; its right adjoint `withUtility` adjoins a utility back, and the two compose to the identity in both directions.

Textbook correspondence. This is the strategic-form game with stochastic outcome mechanism — the most general finite strategic-form object: a strategic-form game in the sense of Osborne–Rubinstein [16] Definition 11.1 or Maschler–Solan–Zamir Definition 4.1, with the textbook "deterministic action profile" replaced by a stochastic kernel. A textbook normal-form game with utilities $u_i : A \to \mathbb{R}$ is the special case in which the kernel is the Dirac evaluation $\sigma \mapsto \delta_\sigma$ (and the outcome type is A); this is what the NFG language compiler produces.

Faithfulness. Strict generalization in two directions. *(1) Stochastic outcomes.* The textbook strategic form is deterministic given a profile; we allow $K(\sigma) \in \mathrm{PMF}(\mathcal{O})$ to be any discrete distribution. This subsumes the deterministic case ($K = \delta \circ f$ for a deterministic f) and accommodates languages with chance moves (EFG, MAID, MultiRound, FOSG) without an external lift. *(2) Outcome type.* The textbook outcome is implicit (the profile itself, or a payoff vector); we expose it as a parameter $\mathcal{O}$ so that an EFG outcome (a terminal node in the tree) and a NFG outcome (a joint action) can both inhabit $\mathcal{G}$ without encoding tricks.

Textbook correspondence. (continued) For a worked confirmation that `IsNash` on a normal-form game's compiled $\mathcal{G}$ unfolds to the textbook payoff inequality, see §9.1 (Prisoner's Dilemma).

3.3 Expected utility: `KernelGame.eu`

Anchor: `KernelGame.eu`.

```
noncomputable def eu (G : KernelGame ι) (σ :  Profile G) (who :  ι) :  ℝ :=
   expect (G.outcomeKernel σ) (fun ω ⇒ G.utility ω who)
noncomputable def correlatedEu (G : KernelGame ι)
   (μ :  PMF (Profile G)) (who :  ι) :  ℝ :=
   expect (G.correlatedOutcome μ) (fun ω ⇒ G.utility ω who)
```

Mathematical content. The expected utility of player i under profile σ in kernel game $\mathcal{G}$ is

$$\mathrm{EU}(\mathcal{G}, \sigma, i) \;=\; \sum_{\omega \in \mathcal{O}} K(\sigma)(\omega) \cdot u(\omega)(i), \tag{2}$$

finite when $\mathcal{O}$ is finite. In the general core API, this is the tsum-based expectation over the discrete support, well-defined on bounded utility for arbitrary (e.g. countably infinite) $\mathcal{O}$. The major theorem statements use finite outcomes only where a finite wrapper is more convenient or where a proof still requires finite support; the corresponding `\_of\_bounded` lemmas in `Math.Probability` and `Concepts/MixedExtension` replay the same identities under a $|u| \le C$ hypothesis without `[Finite \Outcome ]`. The correlated variant is the same expectation under the pushforward $K_*\mu$ of a profile distribution μ.

Textbook correspondence. The expected utility of a strategy profile in a strategic-form game with stochastic outcome (vNM [24], Theorem 1; Maschler–Solan–Zamir §5.2). The von Neumann–Morgenstern expected-utility theorem is what *justifies* taking EU as the criterion of preference; the library's IsNashFor predicate (§4.3) makes this preference choice an explicit parameter so the EU criterion can be replaced by another preorder if desired.

Faithfulness. Reformulation. The function `expect` (§2.1) is exactly the discrete sum $\sum d(\omega)\, f(\omega)$, finite when the outcome carrier is finite and a tsum under boundedness in the general PMF( )case. No measurability obligation arises because PMF( )is the discrete monad.

3.4 Mixed extension: `KernelGame.mixedExtension`

Anchor: `KernelGame.mixedExtension`.

```
open Classical in
noncomputable def mixedExtension (G : KernelGame ι)
    [Fintype ι] :  KernelGame ι where
    Strategy := fun i ⇒ PMF (G.Strategy i)
    Outcome := G.Outcome
    utility := G.utility
    outcomeKernel := fun σ ⇒ (Math.PMFProduct.pmfPi σ).bind G.outcomeKernel
```

Mathematical content. The mixed extension $\mathcal{G}^{\mathrm{mix}}$ of a kernel game $\mathcal{G}$ has strategies $\mathrm{PMF}(\mathsf{S}_i)$ for each player, the same outcome and utility, and the outcome kernel

$$K^{\mathrm{mix}}(\sigma) \;=\; \mathrm{pmfPi}(\sigma) \gg\!= K \;=\; \sum_{x \in \prod_i \mathsf{S}_i} \Big(\prod_i \sigma_i(x_i)\Big) K(x).$$

Independence of the per-player mixing is encoded in `pmfPi`; the original kernel K is then applied to the joint pure profile drawn from this product.

The surrounding mixed-extension lemmas follow the same boundary: EU decomposition, unilateral mixed updates, zero weighted gain, the pure-deviation characterization of Nash, and the mixed-Nash support lemma all work over arbitrary strategy carriers under a bounded-utility hypothesis. When $\mathcal{O}$ is finite, boundedness is derived automatically, so the corresponding finite-outcome wrappers do not require finite strategy carriers.

Textbook correspondence. The mixed extension of a finite game (Nash [15]; Osborne–Rubinstein Definition 32.1; Myerson §3.2): each player randomizes independently over their pure strategies, and payoffs are the expectation under the resulting product distribution.

Faithfulness. Faithful. The displayed formula is exactly the textbook one for finite normal-form games; the only generalization is that K may itself be stochastic (chance moves), in which case the mixed extension's outcome kernel is the composition of the mixing distribution with K. The classical case ($K = \delta$) recovers the textbook formula verbatim.

3.5 Reindexing and `ofEU`: bunched

```
noncomputable def reindex {ι κ : Type} (e : ι ≃ κ)
    (G : KernelGame κ) : KernelGame ι

noncomputable def KernelGame.ofEU
    (Strategy : ι → Type)
    (eu : (∀ i, Strategy i) → Payoff ι) : KernelGame ι
```

`reindex` transports a kernel game across a bijection of player types; this is presentation bookkeeping used by bridges (§6) that canonicalize $\mathcal{I}$ to $\mathrm{Fin}\, n$. `ofEU` constructs a kernel game directly from a per-profile, per-player real-valued function $\mathrm{eu} : \prod_i \mathsf{S}_i \to \mathcal{I} \to \mathbb{R}$ by taking the outcome type to be $\mathcal{I} \to \mathbb{R}$ itself, the kernel to be $\sigma \mapsto \delta_{\mathrm{eu}(\sigma)}$, and the utility to be the identity. The `ofEU`-form is the natural target of the Bayesian-game compilation (§8.1); structural lemmas in `Concepts/OfEUProperties.lean` record that $\mathtt{eu}\,(\mathtt{ofEU}\,\mathsf{S}\,\mathtt{u})\,\sigma\,i = \mathtt{u}\,\sigma\,i$ and that solution-concept predicates on `ofEU`-games unfold to direct EU comparisons.

3.6 Morphisms, simulations, isomorphisms

```
structure ProtocolMap (F F' : GameForm ι) where
    stratMap : ∀ i, F.Strategy i → F'.Strategy i
    outcomeMap : F.Outcome → F'.Outcome
    outcomeKernel_preserved : ∀ σ : F.Profile,
        F'.outcomeKernel (fun i ⇒ stratMap i (σ i)) =
            (F.outcomeKernel σ).bind (fun ω ⇒ PMF.pure (outcomeMap ω))

def ProtocolMap.id (F : GameForm ι) : ProtocolMap F F
def ProtocolMap.comp (g : ProtocolMap F' F'') (f : ProtocolMap F F') : ProtocolMap F F''
```

Mathematical content. A *protocol map* from $\mathcal{F}$ to $\mathcal{F}'$ is a triple $(\mathrm{stratMap}, \mathrm{outcomeMap}, \mathrm{preserved})$ satisfying the *outcome-kernel preservation* law: pushing a profile of $\mathcal{F}$ through stratMap and then evaluating K' produces the same distribution as evaluating K first and pushing the outcome through outcomeMap. Equivalently, stratMap and outcomeMap form a coalgebra-style commuting square in the discrete Giry monad. Identity and composition exist and satisfy the standard laws (`ProtocolMap.id_comp`, `comp_id`, `comp_assoc`).

Variants. `KernelGame.Morphism` (`Core/GameMorphism.lean`) is a protocol map together with utility-preservation $u'(\mathrm{outcomeMap}(\omega))(i) = u(\omega)(i)$. A `KernelGame.Simulation` is a kernel-game morphism (the names coincide). A `KernelGame.Bisimulation` is a simulation whose strategy and outcome maps are bijective with simulation inverse; it expresses an EU-isomorphism. A `GameForm.ProtocolIsomorphism` is the analogous bijective protocol-only version. All of these come with `id` and `comp` together with the corresponding identity/associativity lemmas. Distribution-preserving morphisms can be upgraded to EU-preserving morphisms either from finite outcomes or from explicit bounded-utility hypotheses via `toEUMorphismOfBounded`.

Faithfulness. The morphism vocabulary is project-internal: no textbook canonizes "protocol map between strategic-form games." The categorical laws are proved as `simp`-lemmas (`Core/GameForm.lean:307âĂŞ349`, `Core/GameMorphism.lean:67âĂŞ77`); their content is the distributional analogue of a coalgebra homomorphism.

4 Solution concepts

This section catalogues the solution-concept predicates. The deviation-family schema of §4.1 is the organizing principle: every equilibrium notion is an instance of `IsDeviationEqFamilyFor` for an explicit choice of $\mathcal{D}$. We anchor at IsNash (§4.3) and IsCorrelatedEq (§4.4); other concepts are bunched.

4.1 Deviation-family schema: `Core/GameForm.lean`

```
namespace GameForm
def NoProfitableDeviationFor (F : GameForm ι)
   (pref : ι → PMF F.Outcome → PMF F.Outcome → Prop)
   (who : ι) (statusQuo : PMF F.Outcome)
   {D : Type} (deviate : D → PMF F.Outcome) : Prop :=
      ∀ d : D, pref who statusQuo (deviate d)
structure ProfileDeviationFamily (F : GameForm ι) where
   Dev : ι → Type
   deviate : PMF F.Profile → (∀ who : ι, Dev who → PMF F.Profile)
def IsDeviationEqFamilyFor (F : GameForm ι)
   (pref : ι → PMF F.Outcome → PMF F.Outcome → Prop)
   (μ : PMF F.Profile) (Δ : ProfileDeviationFamily F) : Prop :=
      ∀ who : ι, F.NoProfitableProfileDeviationFor pref who μ (Δ.deviate μ who)
noncomputable def constantDeviationProfileFamily (F : GameForm ι) : ProfileDeviationFamily F
noncomputable def recommendationDeviationFamily (F : GameForm ι) : ProfileDeviationFamily F
```

Mathematical content. A *deviation family* for player i is a type D_i of deviations and a function transforming a profile distribution into a deviated profile distribution given a deviation. A profile distribution μ is a $\mathcal{D}$-equilibrium for preference $\succeq$ when no player has a profitable deviation in $\mathcal{D}$:

$$\forall i, \forall d \in D_i. \quad \succeq_i \big(\mathcal{F}.\text{correlatedOutcome}(\mu),\, \mathcal{F}.\text{correlatedOutcome}(\text{deviate}_i(\mu, d))\big).$$

The two named families are $\mathcal{D}^{\text{const}}$ (deviations are constant strategies; `constantDeviationProfileFamily`) and $\mathcal{D}^{\text{dep}}$ (deviations are functions of the recommendation; `recommendationDeviationFamily`).

Textbook correspondence. The deviation-family schema is project-internal but the two families are textbook: unilateral deviation by a fixed strategy (Nash, CCE) and recommendation-dependent deviation (Aumann's correlated equilibrium [1]).

4.2 Deviation-simulation transport: `Concepts/DeviationSimulation.lean`

Anchor: `OutcomeView`.

```
structure OutcomeView (F : GameForm ι) (Ω : Type) where
   observe : F.Outcome → Ω
def observedPref (V : OutcomeView F Ω)
   (prefOmega : ι → PMF Ω → PMF Ω → Prop) :
   ι → PMF F.Outcome → PMF F.Outcome → Prop :=
      fun who d e ⇒ prefOmega who (PMF.map V.observe d) (PMF.map V.observe e)
```

Mathematical content. An *outcome view* records what part of a game's native outcome is behaviorally relevant for a transport theorem. If one presentation returns terminal payoffs and another returns a richer execution trace, both can be compared after projection to a common carrier Ω. The induced preference `observedPref V prefOmega` compares native outcome laws only through their pushforwards along `V.observe`. This is deliberately more general than expected utility; the EU convenience `observedEuPref` is just the instance where `prefOmega` orders laws by expectation of a payoff function on Ω.

```
structure ProfileRealization (G H : GameForm ι) (Ω : Type) where
  viewG : OutcomeView G Ω
  viewH : OutcomeView H Ω
  rel : G.Profile → H.Profile → Prop
  law_eq : ∀ {σ : G.Profile} {τ : H.Profile}, rel σ τ →
    viewG.law σ = viewH.law τ
structure NashDeviationSimulation (G H : GameForm ι) (Ω : Type)
  extends ProfileRealization G H Ω where
  simulate_target_deviation :
    ∀ {σ : G.Profile} {τ : H.Profile}, rel σ τ →
    ∀ (who : ι) (sH : H.Strategy who),
    ∃ sG : G.Strategy who,
    viewG.law (Function.update σ who sG) =
    viewH.law (Function.update τ who sH)
```

Mathematical content. `ProfileRealization` is a relation, not a map, because many semantic bridges are existential or many-to-many: a behavioral profile may be realized by several mixed profiles, and a compiled trace profile may forget administrative choices. The base law `law_eq` says only that related profiles induce the same observed status-quo law. `NashDeviationSimulation` adds the missing equilibrium-relevant hypothesis: every unilateral deviation available in the target presentation must be matched by some unilateral deviation in the source presentation with the same observed law.

```
theorem NashDeviationSimulation.target_nash_of_source_nash
  (S : NashDeviationSimulation G H Ω)
  (hrel : S.rel σ τ)
  (hN : G.IsNashFor (observedPref S.viewG prefOmega) σ) :
    H.IsNashFor (observedPref S.viewH prefOmega) τ
```

Mathematical content. The proof is the two-line argument one expects. Given a target deviation s_H, use `simulate_target_deviation` to obtain a source deviation s_G. The source Nash hypothesis compares the source status quo with s_G; `law_eq` rewrites the status quo to the target law, and the simulation law rewrites the deviated law to the target deviation.

```
structure ProfileDistributionRealization (G H : GameForm ι) (Ω : Type) where
  viewG : OutcomeView G Ω
  viewH : OutcomeView H Ω
  rel : PMF G.Profile → PMF H.Profile → Prop
  law_eq : ∀ {μG : PMF G.Profile} {μH : PMF H.Profile}, rel μG μH →
    viewG.correlatedLaw μG = viewH.correlatedLaw μH
structure DeviationFamilySimulation
  (G H : GameForm ι) (Ω : Type)
  (ΔG : ProfileDeviationFamily G) (ΔH : ProfileDeviationFamily H)
  extends ProfileDistributionRealization G H Ω where
  simulate_target_deviation :
```

```
∀ {μG : PMF G.Profile} {μH : PMF H.Profile}, rel μG μH →
∀ (who : ι) (dH : ΔH.Dev who),
∃ dG : ΔG.Dev who,
viewG.correlatedLaw (ΔG.deviate μG who dG) =
viewH.correlatedLaw (ΔH.deviate μH who dH)
```

Mathematical content. The family-level version moves from profiles to profile distributions, because correlated and coarse correlated equilibrium are predicates on a correlation device μ. The theorem `DeviationFamilySimulation.target_eq_of_source_eq` transports `IsDeviationEqFamilyFor`; the CE and CCE wrappers instantiate Δ with `recommendationDeviationFamily` and `constantDeviationProfileFamily`. A two-way `DeviationFamilyBisimulation` packages simulations in both directions and yields an iff.

Faithfulness. No finiteness assumptions appear in these transport theorems. They use only exact equality of pushed-forward `PMF` laws. Finite outcome hypotheses remain localized to existence theorems and to expected-value lemmas that need finite sums.

4.3 Nash equilibrium: `KernelGame.IsNash`

Anchor: `KernelGame.IsNash`.

```
open Classical in
def KernelGame.IsNash (G : KernelGame ι) (σ : Profile G) : Prop :=
  ∀ (who : ι) (s' : G.Strategy who),
    G.eu σ who ≥ G.eu (Function.update σ who s') who

def KernelGame.IsNashFor (G : KernelGame ι)
  (pref : ι → PMF G.Outcome → PMF G.Outcome → Prop)
  (σ : Profile G) : Prop :=
    G.toGameForm.IsNashFor pref σ

theorem KernelGame.IsNash_iff_IsNashFor_eu (G : KernelGame ι) (σ : Profile G) :
  G.IsNash σ ↔ G.IsNashFor G.euPref σ
```

Mathematical content. A profile σ is a Nash equilibrium of $\mathcal{G}$ when no player has a strictly improving unilateral deviation:

$$\forall i \in \mathcal{I},\, \forall s' \in \mathsf{S}_i. \quad \mathrm{EU}(\mathcal{G}, \sigma, i) \;\geq\; \mathrm{EU}(\mathcal{G}, \sigma[i \mapsto s'], i). \tag{3}$$

The library's `IsNashFor` variant (`Core/GameForm.lean`) generalizes (3) to take an arbitrary preorder $\succeq$ on $\mathrm{PMF}(\mathcal{O})$:

$$\forall i,\, \forall s'. \quad \succeq_i \big(K(\sigma),\; K(\sigma[i \mapsto s'])\big).$$

The bridge `IsNash_iff_IsNashFor_eu` certifies that IsNash is `IsNashFor` instantiated at the EU preference $\succeq^{\mathrm{EU}}$.

Textbook correspondence. Osborne–Rubinstein Definition 14.1 (Nash equilibrium of a strategic game): a strategy profile a^* such that for every i and every $a_i \in A_i$, $u_i(a^*_{-i}, a^*_i) \geq u_i(a^*_{-i}, a_i)$. Maschler–Solan–Zamir Definition 5.1 is identical with notation σ for the profile. Myerson Definition 3.1 is the same modulo the use of $\gtrsim$ in place of $\geq$ on real numbers. (3) is verbatim these definitions specialized to the EU preorder, with `Function.update` encoding the "a^*_{-i}, a_i" notation explicitly: `Function.update $\sigma $ who s'` is the profile that agrees with σ at every coordinate except `who`, where it is replaced by `s'`.

Faithfulness. Faithful, with strict generalization through `IsNashFor`. The weak-preference convention (the textbook uses $\geq$, the library uses $\geq$ on EU; `IsStrictNash` below uses $>$) matches the weak Nash equilibrium of Osborne–Rubinstein 14.1. See §9.1 for an end-to-end unfolding on Prisoner's Dilemma: the predicate evaluates to four real inequalities, one for each player and each alternative action, exactly as in the textbook treatment.

Variants. The bunched siblings of `IsNash` are:

- `IsBestResponse(For)` — a single coordinate is a best response against fixed opponents (the existential factor of Nash).
- `IsStrictNash(For)` — strict inequality for every $s' \neq \sigma_i$.
- `IsDominant(For)` / `IsStrictDominant(For)` — the coordinate is best-response against *every* opponent profile.
- `WeaklyDominates(For)` / `StrictlyDominates(For)` — pairwise dominance as a binary predicate on strategies.

Each is the obvious quantifier rearrangement of (3) and unfolds analogously. The dominance⇒Nash chain (`KernelGame.dominant_is_nash`) and Nash⇔best-response characterization are recorded as one-line corollaries.

4.4 Correlated equilibrium: `KernelGame.IsCorrelatedEq`

Anchor: `KernelGame.IsCorrelatedEq`.

```
def IsCorrelatedEq (G : KernelGame ι) (μ :  PMF (Profile G)) : Prop := by
   classical
   exact ∀ (who :  ι) (dev :  G.Strategy who → G.Strategy who),
      G.correlatedEu μ who ≥
      G.correlatedEu (G.unilateralDeviationDistribution μ who dev) who
def IsCoarseCorrelatedEq (G : KernelGame ι) (μ :  PMF (Profile G)) : Prop := by
   classical
   exact ∀ (who :  ι) (s' :  G.Strategy who),
      G.correlatedEu μ who ≥
      G.correlatedEu (G.constantDeviationDistribution μ who s') who
```

Mathematical content. A profile distribution $\mu \in \mathrm{PMF}(\prod_i \mathsf{S}_i)$ is a *correlated equilibrium* of $\mathcal{G}$ when, viewing μ as a public correlation device privately recommending action μ_i to each player, no player gains by replacing their recommendation by any function of it:

$$\forall i,\, \forall \mathrm{dev} : \mathsf{S}_i \to \mathsf{S}_i. \quad \mathbb{E}_{\sigma\sim\mu}\big[u_i(K(\sigma))\big] \;\geq\; \mathbb{E}_{\sigma\sim\mu}\big[u_i(K(\sigma[i \mapsto \mathrm{dev}(\sigma_i)]))\big].$$

Coarse correlated equilibrium [4] restricts dev to constant functions dev $\equiv s'$; this corresponds to the deviation family $\mathcal{D}^{\mathrm{const}}$ of §4.1.

Textbook correspondence. Aumann [1] Definition 2 (correlated equilibrium of a finite Bayesian game with correlation device): for every recommended action a, the conditional expectation under μ of deviating to any $a' \neq a$ is no larger than the conditional expectation of complying. Our predicate is the (well-known) equivalent ex-ante formulation: summing over all recommendations of the conditional inequality gives (3)-style total inequality on μ, and the converse holds because the conditional inequalities can be recovered by deviation functions that act as the identity outside the conditioning event.

Faithfulness. Faithful, with reformulation. The library's IsCorrelatedEq is the ex-ante form; the equivalent ex-post (conditional) form is `Concepts/CorrelatedEqProperties.lean`'s `isCorrelatedEq_iff_pointwise` (and similarly for the preference-parameterized variant). Existence is the content of the main chapter's CE existence theorem (`KernelGame.correlatedEq_exists`); the bounded-utility sibling `KernelGame.correlatedEq_exists_of_bounded` removes finite outcomes while retaining the finite-strategy Brouwer hypothesis.

Variants. `IsCorrelatedEqFor`, `IsCoarseCorrelatedEqFor` are the preference-parameterized versions; `IsCorrelatedEqFor.toCoarseCorrelatedEqFor` records the inclusion CE $\subseteq$ CCE; the schema instantiation $\mu = \delta_\sigma$ recovers IsNashFor from IsCoarseCorrelatedEqFor.

4.5 Other solution concepts: bunched

We document the remaining concepts in groups. Each group cites its anchor textbook reference; the structural relation to IsNash or IsCorrelatedEq is recorded in the prose.

Pareto and welfare. `Concepts/NashPareto.lean`, `Concepts/WelfareTheorems.lean`, `Concepts/PriceOfAnarchy.lean`. `ParetoDominatesFor` is the binary preference-parameterized predicate "σ Pareto-dominates τ"; `IsParetoEfficient(For)` is its $\neg\exists$-closure. `socialWelfare` is $\sum_i \mathrm{EU}(\mathcal{G}, \sigma, i)$, requiring a finite player set. `optimalWelfare`, `worstNashWelfare`, `bestNashWelfare` are the standard sup/inf over profile sets; the Lean library does not yet package the ratio-valued `PoA`/`PoS` definitions ([7, 17]).

Security and individual rationality. `Concepts/SecurityStrategy.lean`, `Concepts/IndividualRationality.lean`. `securityLevel` is the finite attained version of $\sup_s \inf_{\sigma_{-i}} \mathrm{EU}(\mathcal{G}, \sigma_{-i}[i \mapsto s], i)$, while `securityLevelSup` and `worstCaseEUInf` are the order-theoretic sup/inf versions for non-enumerated carriers. A security strategy attains the finite supremum. Individual rationality says no Nash payoff is below the relevant security level (Maschler–Solan–Zamir §5.4).

Regret. `Concepts/Regret.lean`. External regret of σ for i is $\max_{s'}(\mathrm{EU}(\sigma[i \mapsto s'], i) - \mathrm{EU}(\sigma, i))^+$; internal regret replaces s' by a function of σ_i. The regret-equilibrium duality (zero average external regret iff CCE; zero average internal regret iff CE) is recorded as `regret_zero_iff_isCoarseCorrelatedEq` and its CE analogue.

Approximate equilibria. `Concepts/ApproximateNash.lean`. `IsEpsilonNash $\varepsilon $` relaxes the Nash inequality by an additive ε slack; $\varepsilon = 0$ recovers `IsNash`. Closure under ε-relaxation extends to the deviation-family schema, hence applies to ε-CCE and ε-CE.

Potential games. `Concepts/PotentialGame.lean`, `PotentialFIP.lean`, `PotentialWellFounded.lean`, `PotentialTeam.lean`. `IsExactPotential $\Phi $` requires $\mathrm{EU}(\sigma', i) - \mathrm{EU}(\sigma, i) = \Phi(\sigma') - \Phi(\sigma)$ for every unilateral deviation $\sigma \to \sigma'$; `IsOrdinalPotential $\Phi $` replaces equality by sign-equality. `IsExactPotential.toOrdinal` gives the expected implication. `IsOrdinalPotential.nash_of_maximizer`: every Φ-maximizer is Nash. `IsTeamGame.isExactPotential`: team games are exact potential games with $\Phi = u_{\text{any}}$ ([12]).

Zero-sum, constant-sum, team, symmetric, evolutionary. `Concepts/ZeroSum.lean`, `ConstantSum.lean`, `TeamGame.lean`, `SymmetricGame.lean`, `EvolutionaryStability.lean`. `IsZeroSum`: $\sum_i u(\omega)(i) = 0$ for all ω. Constant-sum, team-game, and symmetric predicates are similar. `ZeroSumNash`, `ConstantSumNash` record that two-player zero/constant-sum Nash profiles are exactly the maximin/security profiles. `IsEvolutionarilyStable` is the strict-dominance condition on a symmetric two-player game ([11]).

Rationalizability and dominance solvability. `Concepts/Rationalizability.lean`, `Concepts/DominanceSolvable.lean`, `Concepts/DominanceSolvability.lean`. `Survives n` is the predicate "survives n rounds of iterated strict dominance elimination," defined inductively; `IsRationalizable` is the inf over n. `IsNash.isRationalizable` and `IsDominant.isRationalizable` are the expected implications. `Concepts/DominanceSolvable.lean` records that the unique surviving profile (when one exists) is Nash.

Common knowledge. `Concepts/CommonKnowledge.lean`. Aumann's common-knowledge operator on a finite partition structure ([2]); used as scaffolding for the mechanism layer.

5 Languages

Each of the six languages in `Languages/` ships with a `Syntax` file (the AST), often an `SOS` operational semantics, and a `Compile` file producing a `KernelGame`. This section anchors at the compilation *shape* for each language; full Syntax content beyond the top-level record is bunched.

5.1 NFG: `Languages/NFG/Compile.lean`

Anchor: `NFG.NFGGame.toKernelGame`.

```
structure NFGGame (ι :  Type) (A : ι → Type) where
   Outcome :  Type
   outcome :  (∀ i, A i) → Outcome
   utility :  (∀ i, A i) → ι → ℝ
noncomputable def NFGGame.toKernelGame (G : NFGGame ι A) : KernelGame ι where
   Strategy := A
   Outcome := G.Outcome
   utility := fun ω i ⇒ ...
   outcomeKernel := fun σ ⇒ PMF.pure (G.outcome σ)
```

Mathematical content. A normal-form game over a finite player set $\mathcal{I}$ and action sets $\{A_i\}_i$ is the textbook tuple $(\mathcal{I}, \{A_i\}_i, u)$. The compiler takes the strategy type to be A_i, the outcome type to be a chosen $\mathcal{O}$ with a deterministic projection from joint actions, and the kernel to be the Dirac evaluation $\sigma \mapsto \delta_{G.\mathrm{outcome}(\sigma)}$. The compiled $\mathcal{G}$ is deterministic; randomness enters only at the mixed extension. The structure does not impose finiteness on the action types A_i — countably infinite action spaces are admitted at the definitional level (see `Languages/NFG/CountableExample.lean` for an $\mathbb{N}$-valued smoke test); the mixed-Nash existence theorem reintroduces `[Fintype (A i)]` where the proof requires it.

Textbook correspondence. Osborne–Rubinstein Definition 11.1 (strategic-form game). The compilation is the identity on the strategic form modulo the choice of outcome type.

Faithfulness. Faithful, with reformulation: the textbook treats utility as a direct function of profile; we route through an outcome type so that a kernel-game-level theorem applies uniformly to NFGs and to languages with non-trivial outcome types.

Variants. `Languages/NFG/` also packages worked subclasses, each as its own `NFGGame` construction: `CongestionGame.lean` (Rosenthal's potential), `CoalitionalGame.lean` (the Shapley value definition plus null-player, additivity, linearity, and symmetry lemmas), `Bargaining.lean` (Nash bargaining), `Matching.lean` (stable-matching predicates), `PublicGoods.lean`, `Stackelberg.lean`. Each subclass lives in its own namespace and exposes its characteristic theorem.

5.2 EFG: `Languages/EFG/`

Anchor: `EFG.GameTree, EFG.evalDist, EFG.toKernelGame.`

```
structure InfoStructure where
   n :  Nat
   Infoset :  Fin n → Type
   arity :  (p :  Fin n) → Infoset p → Nat
   arity_pos :  ∀ p I, 0 < arity p I
inductive GameTree (S : InfoStructure) (Outcome :  Type) where
   | terminal (z :  Outcome)
   | chance (k :  Nat) (μ :  PMF (Fin k)) (hk :  0 < k) (next :  Fin k → GameTree S Outcome)
   | decision {p :  S.Player} (I : S.Infoset p) (next :  S.Act I → GameTree S Outcome)
structure EFGGame where
   inf :  InfoStructure ; Outcome :  Type
   tree :  GameTree inf Outcome ; utility :  Outcome → Payoff inf.Player
noncomputable def GameTree.evalDist {S : InfoStructure} {Outcome :  Type}
   (σ :  BehavioralProfile S) : GameTree S Outcome → PMF Outcome
   | .terminal z ⇒ PMF.pure z
   | .chance _ μ _ next ⇒ μ.bind (fun b ⇒ (next b).evalDist σ)
   | .decision (p := p) I next ⇒ (σ p I).bind (fun a ⇒ (next a).evalDist σ)
```

Mathematical content. An extensive-form game is a triple $(\mathsf{InfoStruct}, \mathsf{Tree}, u)$ where $\mathsf{InfoStruct}$ records the players, their information sets, and the action arity at each information set; the tree is inductively built from terminal, chance, and decision nodes; and the utility maps terminal outcomes to payoff vectors. Strategies come in three flavors: pure $\Sigma_p^{\text{pure}} = \forall I, \mathrm{Fin}(\mathrm{arity}(p, I))$, behavioral $\Sigma_p^{\text{beh}} = \forall I, \mathrm{PMF}(\mathrm{Fin}(\mathrm{arity}(p, I)))$, and mixed $\Sigma_p^{\text{mix}} = \mathrm{PMF}(\Sigma_p^{\text{pure}})$. The function `evalDist` evaluates the tree under a behavioral profile to a $\mathrm{PMF}(\mathcal{O})$ by structural recursion: chance binds the chance distribution, decision binds the player's behavioral distribution at the relevant information set, terminal returns Dirac. The compiled kernel game has `Strategy` the behavioral type, the EFG outcome as outcome, `evalDist` as kernel, and the EFG utility as utility.

Textbook correspondence. Kuhn's representation [9]; Osborne–Rubinstein Chapter 6. The `InfoStructure` factoring through information-set IDs (rather than directly through a player type) is a presentation choice for the type theory: a decision node stores only its information-set ID, and the player is recovered from the information structure. Augmented EFGs in the FOSG-paper sense [8] are a thin wrapper adding public and per-player history partitions (`Languages/EFG/Augmented.lean`).

Faithfulness. Faithful, with reformulation: distributional semantics rather than deterministic-with-lift. See chapter §3.2 for the rationale. Subgame-perfect equilibrium

(`EFG.EFGGame.IsSubgamePerfectEq` in `Languages/EFG/Refinements.lean`) is defined as a Nash equilibrium of every subgame; the ODP characterization (§7.4) makes this a one-shot deviation check.

Variants. `Languages/EFG/` contains `Augmented.lean` (the augmented EFG), `SOS.lean` (operational semantics for the tree), `Compile.lean` and `CompileObs.lean` (the latter compiling to ObsModelCore for Kuhn), `Kuhn.lean` (language-level Kuhn specialization), `Refinements.lean` (subgame perfection, behavioral/pure refinements), `Examples.lean`, `Tests.lean`.

5.3 MAID: `Languages/MAID/`

Anchor: `MAID.MAIDGame.toKernelGame`.

```
structure MAIDGame where
   Players :  Type ; [fin :  Fintype Players]
   Node :  Type
   parents :  Node → Finset Node
   nodeKind :  Node → NodeKind - chance, decision, utility
   domain :  Node → Type ; chanceDist, decisionInfo, utilityFn fields
   acyclic :  ...
noncomputable def MAIDGame.toKernelGame (G : MAIDGame) :  KernelGame G.Players
```

Mathematical content. A multi-agent influence diagram [6] is a directed acyclic graph whose nodes are typed as chance, decision, or utility, with edges encoding informational and causal dependence. The compiler implements a *frontier evaluator* that, at each step, picks a node whose parents are all assigned and either samples its chance distribution, consults the player's strategy at the relevant decision node, or accumulates its utility. Order-independence (the diamond lemma in `MAID/FrontierLemmas.lean`) makes the compilation well-defined.

Textbook correspondence. Koller–Milch Definition 1; Pfeffer–Gal Section 2. The Lean formalization restricts to finite-domain MAIDs (each node has a `Fintype` domain). Strategic-relevance and d-separation analyses, prominent in the MAID literature, are not formalized; the order-independence content is.

Faithfulness. Faithful, with restriction to finite-domain nodes. See chapter §5.4 for the rationale on why we accepted the proof obligations of order-independent compilation rather than fixing a topological order.

Variants. `Languages/MAID/` contains `Syntax.lean`, `SOS.lean`, `FrontierEval.lean`, `FrontierLemmas.lean`, `FoldEval.lean`, `Prefix.lean` (subdiagram framework), `Compile.lean`, `CompileObs.lean`, `Kuhn.lean`, `Theorems.lean`.

5.4 MultiRound: `Languages/MultiRound/`

Anchor: `MultiRoundGame, Round`.

```
structure Round (n :  Nat) (S V A Sig :  Type) where
   signal :  S → PMF (Fin n → Sig)
   view :  Fin n → S → Sig → V
   transition :  S → (Fin n → Option A) → S
structure MultiRoundGame (n :  Nat) (S V A Sig :  Type) where
```

```
    init :  S
    rounds :  List (Round n S V A Sig)
noncomputable def MultiRoundGame.toKernelGame
```

Mathematical content. A multi-round game is an initial state and a finite list of rounds [21]. Each round samples a joint signal from a state-dependent distribution, lets each player observe their own state-and-signal-derived view, and applies a deterministic transition keyed on the joint optional action. Strategies are pure and memoryless at the protocol level — a function from view to optional action — with mixed/behavioral and recall structure imposed externally. The compilation generates the outcome kernel by folding the round list and threading the state through the chain of observations and transitions.

Variants. `Languages/MultiRound/RepeatedGame.lean` restricts to identical rounds; `StochasticGame.lean` [20] specializes the transition to be itself stochastic and state-conditioned. `CompileObs.lean` and its linearizations (`CompileObsLin.lean`, `CompileObsLinAdequacy.lean`, `CompileObsLinRecall.lean`) compile to the `ObsModelCore` substrate that the Kuhn theorem (§7.5) is stated against.

5.5 FOSG: `Languages/FOSG/`

Anchor: `FOSG.toKernelGame`.

```
structure FOSG (ι W A : Type) (Obs :  ι → Type) (Pub :  Type) where
    init :  W
    active :  W → Finset ι
    availableActions :  (w :  W) → ι → Set A
    terminal :  W → Prop
    transition :  ...
    reward, privObs, pubObs :  ...
    terminal_active_eq_empty :  ...
    terminal_no_legal, nonterminal_exists_legal :  ...
noncomputable def FOSG.toKernelGame (G : FOSG ...)  [BoundedHorizon] :  KernelGame ι
```

Mathematical content. A factored-observation stochastic game [8] is the most general of the language layers: a state space, an active-set function (the players who must move at each state), per-player available actions, a transition kernel, a reward vector, a per- player private observation, and a public observation. Strategies are functions of observation sequences; histories are recorded by an explicit `History` type. Bounded horizon makes the compilation finite.

Textbook correspondence. Kovařík et al. [8] Definition 1.

Variants. `Languages/FOSG/` contains `Basic.lean`, `History.lean`, `Information.lean`, `Strategy.lean`, `Values.lean`, `Execution.lean`, `OutcomeClosure.lean`, `Serial.lean`, `Compile.lean`, `Kuhn.lean`, `Theorems.lean`, with the canonical `ObsModelCore` bridge in `Theorems/Kuhn/`.

5.6 Intrinsic-form: `Languages/Intrinsic/`

Anchor: `Intrinsic.productMixedToBehavioral`, `Intrinsic.kuhn\_equivalence`.

```
structure WGame where
   P : Type ; owner :  A → P
   owner_nonempty :  ∀ p :  P, ∃ a :  A, owner a = p
   pref :  P → OutcomeLaw toWModel → OutcomeLaw toWModel → Prop
noncomputable def productMixedToBehavioral (G : WGame) :  ...
theorem behavioral_realizes_productMixed (G : WGame) :  ...
def PerfectRecall (G : WGame) (φ :  ConfigOrdering G.toWModel)
   (p :  G.P) : Prop - HDLC Definition 14
theorem kuhn_equivalence
   (G : WGame) (hsolv :  Solvable G.toWModel)
   (φ :  ConfigOrdering G.toWModel)
   (_hcausal :  CausalWith G.toWModel φ) (p :  G.P)
   (hpr :  PerfectRecall G φ p) (μ_p :  MixedStrategy G p) :
   ∃ π_p, KuhnOutcomeEquivalent G hsolv p μ_p π_p
```

Mathematical content. The intrinsic (Witsenhausen-style) representation of a game [5] exposes agent decision *addresses* and information sets directly, without fixing an order in which agents act. Three strategy types are defined: *mixed* (a distribution over a player's joint pure strategy space $\prod_{a \in A_p} \Lambda_a$), *product-mixed* (independent randomization over each address), and *behavioral* (independent randomization at each information set). The unconditional product-mixed/behavioral equivalence (Proposition 13) is the intrinsic-form analogue of one direction of Kuhn's theorem and holds without any recall hypothesis.

For a solvable W-model equipped with a causal ordering, the full outcome-law equivalence (Theorem 16) holds under perfect recall: a player's mixed strategy and the induced product-mixed strategy produce the same outcome distribution against every opponent profile. The formalization strengthens the `PerfectRecall` predicate to encode both clauses of Definition 14 directly on `OrderingPrefix`. Consequently, the prefix-stability fact used to prove measurability of the mixed-to-behavioral construction is derived from perfect recall rather than assumed as an additional hypothesis; the global helper `orderInfoCompatibleWith\_of\_forall\_perfectRecall` packages the same consequence when all players satisfy perfect recall. The final theorem statement therefore matches the paper at the information-structure level: causality plus perfect recall, with solvability supplied explicitly for the outcome map.

Textbook correspondence. Heymann–De Lara–Chancelier 2020 Definitions 8, 10, 11, 14; Propositions 12, 13; Theorem 16. The intrinsic layer is structurally a separate syntactic root than the EFG/MAID/MR/FOSG family.

6 Bridges

Bridges between languages are commuting-compilation theorems; the expressiveness lens of `Languages/Expressiveness/` packages them in a uniform shape.

6.1 Languages and reductions: anchor

Anchor: `Expressiveness.Language, Expressiveness.Reduction.`

```
namespace GameTheory.Languages.Expressiveness
structure Language (ι :  Type) where
   Syntax :  Type u
   compile :  Syntax → KernelGame ι
```

```
structure Reduction {ι :  Type} (L : Language ι) (M : Language ι)
   (R : KernelGame ι → KernelGame ι → Prop) where
   translate :  L.Syntax → M.Syntax
   sound :  ∀ x :  L.Syntax, R (L.compile x) (M.compile (translate x))
```

Mathematical content. A *language* for a fixed player type $\mathcal{I}$ is a syntax class together with a compilation $\rightsquigarrow$: Syntax $\to \mathcal{G}_{\mathcal{I}}$. A *reduction* from L to M over a semantic relation R is a translation on syntax together with a proof that the compiled games are R-related for every source program. Reductions compose under transitivity of R.

Variants. The relation taxonomy in `Expressiveness/Relations.lean` catalogues: ProtocolEquivalent (game-form-level isomorphism, ignoring utility); UtilityDistributionEquivalent (bisimulation; both protocol and joint utility distribution preserved); UtilityDistributionSimulation (the directed version); EUEquivalent (equal expected utility for each player at corresponding profiles); EUSimulation (directed EU-comparison; equilibria of the source pull back to the target). The bridges of the next subsection are reductions in this lens.

6.2 Concrete bridges: bunched

NFG → EFG (`Languages/Bridges/NFG_EFG.lean`). Each finite NFG is presented as a depth-1 EFG with a single information set per player and a flat tree of $|\mathsf{A}_i|$ branches at each. Soundness at the EU-isomorphism level.

NFG → FOSG (`Languages/Bridges/NFG_FOSG.lean`). Each finite NFG is presented as a one-shot FOSG with two world states ($\mathsf{init}, \mathsf{terminal}$), every player active at init, no observations, and a single deterministic transition delivering the NFG payoff as the FOSG reward. Soundness: `NFGGame.toFOSG_terminal_iff` and the corresponding EU equivalence.

EFG → NFG (`Languages/Bridges/EFG_NFG.lean`). The strategic form of a finite EFG: pure strategy = function from information set to action, payoff = expected payoff under the behavioral lift of that pure strategy. Soundness at the EU-equivalence level.

MAID → EFG (`Languages/Bridges/MAID_EFG.lean`). A bounded MAID compiles to an EFG via a topological scheduling of its decision nodes; soundness at the strategic-form refinement level (information sets preserved).

EFG → FOSG (`Languages/Bridges/FOSG/`). `ObsModelCore.lean`, `AugmentedEFG.lean`, `SerialExec.lean`. An EFG with serial execution is presented as an FOSG. Soundness at the game-form simulation level.

6.3 Solution-concept transport

The main chapter's map-based transport theorem (`Core/UtilityInvariance.lean`) states that a protocol map with bijective `stratMap` and bijective `outcomeMap` transports the preference-parameterized Nash, best-response, dominance, weak/strict dominance, correlated-equilibrium, and coarse-correlated-equilibrium predicates pointwise across the bijection on profiles. Specializing to a kernel-game simulation (utility-preserving) yields the EU-specific transport laws.

The observed-law transport layer (`Concepts/DeviationSimulation.lean`) removes the need for an outcome bijection. It compares two games through views into a common observed carrier Ω, relates profiles or profile distributions by equality of the pushed-forward outcome law, and requires one-way simulation of target deviations by source deviations. Under these hypotheses, Nash and arbitrary `ProfileDeviationFamily` equilibria transfer from source to target; a two-way simulation gives an iff. These results are the operational content of bridges whose target presentation contains extra trace data or serialized administrative structure: a Nash equilibrium of the abstract source remains Nash after compilation provided all compiled unilateral deviations are observed already by some source-side unilateral deviation.

7 Major theorems

7.1 Mixed Nash existence: `KernelGame.mixed_nash_exists_of_bounded`

Anchor: `mixed_nash_exists_of_bounded, mixed_nash_exists.`

```
theorem KernelGame.mixed_nash_exists_of_bounded (G : KernelGame ι)
    [Fintype ι] [DecidableEq ι]
    [∀ i, Finite (G.Strategy i)] [∀ i, Nonempty (G.Strategy i)]
    {C : ι → ℝ} (hbd : ∀ who ω, |G.utility ω who| ≤ C who) :
    ∃ σ : ∀ i, PMF (G.Strategy i), G.mixedExtension.IsNash σ
theorem KernelGame.mixed_nash_exists (G : KernelGame ι)
    [Fintype ι] [DecidableEq ι]
    [∀ i, Finite (G.Strategy i)] [∀ i, Nonempty (G.Strategy i)]
    [Finite G.Outcome] :
    ∃ σ : ∀ i, PMF (G.Strategy i), G.mixedExtension.IsNash σ
```

Mathematical content. Every finite-strategy game with bounded player utilities has a mixed Nash equilibrium. Hypotheses: finite player type, decidable equality on players, finite non-empty strategy space per player, and a playerwise bound $|u(\omega, i)| \le C_i$. The outcome carrier need not be finite. The theorem `mixed_nash_exists` is the finite-outcome wrapper that derives boundedness automatically.

Proof structure. The proof follows Nash's gain-map argument [15]. Define $\mathrm{gain}_i(\sigma, s) = \mathrm{EU}(\mathcal{G}^{\mathrm{mix}}, \sigma[i \mapsto \delta_s], i) - \mathrm{EU}(\mathcal{G}^{\mathrm{mix}}, \sigma, i)$, the gain to player i from deviating to pure s, and

$$\Phi_i(\sigma)(s) \;=\; \frac{\sigma_i(s) + (\mathrm{gain}_i(\sigma, s))^+}{1 + \sum_{s'} (\mathrm{gain}_i(\sigma, s'))^+}.$$

Φ is a continuous self-map of the product simplex $\prod_i \Delta(\mathsf{S}_i)$. Brouwer's fixed-point theorem (mathlib's `brouwerFixedPoint` on a closed disk, lifted to the product simplex via the homeomorphism in `Concepts/ProductSimplexBrouwer.lean`) gives a fixed point σ^*. The algebraic step in `Theorems/NashExistenceMixed.lean` (section `NashMapAlgebra`) shows that a fixed point of Φ is a Nash equilibrium: the positive-part bracketing forces every player's gain to vanish at σ^*.

Textbook correspondence. Nash 1951 Theorem 1; Osborne–Rubinstein Proposition 33.1.

Faithfulness. Faithful. The library is honest about non-constructivity: `mixed_nash_exists_of_bounded` is `noncomputable` because Brouwer is. A constructive route via Sperner's lemma is listed in chapter §10 as a future direction.

7.2 Correlated and coarse correlated equilibrium existence

```
theorem KernelGame.correlatedEq_exists_of_bounded (G : KernelGame ι) [...]
theorem KernelGame.correlatedEq_exists (G : KernelGame ι) [...]  theorem
KernelGame.coarseCorrelatedEq_exists_of_bounded (G : KernelGame ι) [...]  theorem
KernelGame.coarseCorrelatedEq_exists (G : KernelGame ι) [...]
```

The independent product $\bigotimes_i \sigma_i^*$ over the mixed Nash profile supplied by the main chapter's mixed-existence theorem is a correlated equilibrium of the original $\mathcal{G}$; this is the route taken by `Theorems/CorrelatedEqExistence.lean`. CCE existence is the corollary $\succeq$-CE $\subseteq$ $\succeq$-CCE. See chapter §6.2 for the alternative no-regret-learning route, not formalized. The `\_of\_bounded` statements require finite strategy spaces but not finite outcomes.

7.3 Minimax theorem: `KernelGame.von_neumann_minimax_of_bounded`

Anchor: `von_neumann_minimax_of_bounded`, `von_neumann_minimax`.

```
theorem KernelGame.von_neumann_minimax_of_bounded (G : KernelGame (Fin 2))
   [∀ i, Finite (G.Strategy i)] [∀ i, Nonempty (G.Strategy i)]
   (hzs :  G.IsZeroSum)
   {C :  Fin 2 → ℝ} (hbd :  ∀ i ω, |G.utility ω i| ≤ C i) :
   ∃ (v :  ℝ) (σ :  Profile G.mixedExtension),
      G.mixedExtension.IsNash σ ∧ G.mixedExtension.eu σ 0 = v ∧
      (∀ s₁, G.mixedExtension.eu (Function.update σ 1 s₁) 0 ≥ v) ∧
      (∀ s₀, G.mixedExtension.eu (Function.update σ 0 s₀) 0 ≤ v)
theorem KernelGame.von_neumann_minimax (G : KernelGame (Fin 2))
   [Finite G.Outcome] [∀ i, Finite (G.Strategy i)] [∀ i, Nonempty (G.Strategy i)]
   (hzs :  G.IsZeroSum) :
   ∃ (v :  ℝ) (σ :  Profile G.mixedExtension),
      G.mixedExtension.IsNash σ ∧ G.mixedExtension.eu σ 0 = v ∧
      (∀ s₁, G.mixedExtension.eu (Function.update σ 1 s₁) 0 ≥ v) ∧
      (∀ s₀, G.mixedExtension.eu (Function.update σ 0 s₀) 0 ≤ v)
```

Mathematical content. Every finite-strategy two-player zero-sum game with bounded utility has a mixed Nash profile whose player-0 payoff is a saddle value: player 1 cannot lower it by deviating and player 0 cannot raise it by deviating. A separate lemma records that all mixed Nash equilibria yield the same expected utility for player 0. The usual finite-game max-min/min-max equality is the textbook reading of these saddle inequalities, but it is not the literal statement packaged here.

Proof structure. Existence of a mixed Nash profile is `mixed_nash_exists_of_bounded`. Equality of EU at any two Nash profiles is the saddle-point property of zero-sum Nash, recorded as `IsZeroSum.nash_eu_eq_of_bounded` in `Theorems/Minimax.lean`.

Textbook correspondence. von Neumann 1928 [23]; Maschler–Solan–Zamir Theorem 7.6.

7.4 Backward induction (Zermelo) and the ODP

Anchor: `HasNoOneShotDeviation`, `oneShotDeviation_iff_spe`, `zermelo`.

```
def HasNoOneShotDeviation (G : EFGGame)
   (σ :  PureProfile G.inf) :  Prop :=
   ∀ {p} (I : G.inf.Infoset p) (next :  G.inf.Act I → GameTree G.inf G.Outcome),
      (∃ h, ReachBy h G.tree (.decision I next)) →
```

```
      ∀ a' :  G.inf.Act I,
         expect ((next (σ p I)).evalDist (pureToBehavioral σ)) (G.utility · p) ≥
         expect ((next a').evalDist (pureToBehavioral σ)) (G.utility · p)
theorem oneShotDeviation_iff_spe_of_bounded (G : EFGGame)
   (σ :  PureProfile G.inf) (hpi :  IsPerfectInfo G.tree)
   {C :  G.inf.Player → ℝ} (hbd :  ∀ p ω, |G.utility ω p| ≤ C p) :
   G.IsSubgamePerfectEq σ ↔ HasNoOneShotDeviation G σ
theorem zermelo_of_bounded (G : EFGGame) (hpi :  IsPerfectInfo G.tree)
   {C :  G.inf.Player → ℝ} (hbd :  ∀ p ω, |G.utility ω p| ≤ C p) :
   ∃ σ :  PureProfile G.inf, G.IsSubgamePerfectEq σ
```

Mathematical content. The one-shot deviation principle: in a finite perfect-information EFG, a pure profile is subgame-perfect iff it has no profitable one-shot deviation at any reachable decision node. Zermelo's theorem: every finite perfect-information EFG has a pure subgame-perfect equilibrium, constructed by backward induction. The displayed theorems are the bounded-utility forms; finite outcome wrappers `oneShotDeviation\_iff\_spe` and `zermelo` derive boundedness automatically.

Proof structure. `Theorems/Zermelo.lean`. `perfectInfo_disjoint_subtrees`: in a perfect-info tree, sibling subtrees have disjoint information-set supports, so backward induction can fix actions at one decision node without affecting optimality elsewhere. `exists_noOSD`: the inductive backward construction yields a profile with no profitable one-shot deviation. Zermelo follows from `exists_noOSD` and the ODP.

Textbook correspondence. Zermelo 1913 (the chess theorem); modern treatment in Selten 1965 [18, 19]; Osborne–Rubinstein Chapter 6.4. ODP is folklore (see Fudenberg–Tirole §3.3 for a modern statement).

7.5 Kuhn's theorem: `Theorems/Kuhn/`

Anchor: `ObsModelCore, kuhn_behavioral_to_mixed, kuhn_mixed_to_behavioral_semantic.`

```
structure ObsModelCore (ι σ :  Type) (Obs :  ι → Type)
   (Act :  (i :  ι) → Obs i → Type) where
   - world state, transition kernel, per-player observation, ...
def NoNontrivialInfoStateRepeat (O : ObsModelCore ...)  :  Prop
def HorizonSeparation (O : ObsModelCore ...)  :  Prop
def ActionPosteriorLocal (O : ObsModelCore ...)  :  Prop
def PerStepPlayerRecall (O : ObsModelCore ...)  :  Prop
theorem ObsModelCore.kuhn_behavioral_to_mixed
   (O : ObsModelCore ...)  [...]  (hH : HorizonSeparation O) (hR : NoNontrivialInfoStateRepeat
O) :
   ∀ β :  BehavioralProfile O, runDist O β = (behavioralToMixedJoint β).bind (runDistPure O)
theorem ObsModelCore.kuhn_mixed_to_behavioral_semantic
   (O : ObsModelCore ...)  [...]  (hL : ActionPosteriorLocal O) :
   ∀ μ :  PMF (PureProfile O), ∃ β, runDist O β = μ.bind (runDistPure O)
```

Mathematical content. The library proves Kuhn's behavioral ↔ mixed equivalence at a representation-neutral substrate, the `ObsModelCore` record: a tuple recording the world state, each player's observation type, the per-observation action arity, and a transition kernel. Behavioral

profiles are per-player $\mathsf{InfoState}_i \to \mathrm{PMF}(\mathsf{A}_i)$; mixed profiles are per-player distributions over the deterministic functions $\mathsf{InfoState}_i \to \mathsf{A}_i$.

The four hypothesis predicates control which direction of Kuhn applies:

- *NoNontrivialInfoStateRepeat*: no information state is visited twice along any trace (except at trivially-arity states). This enforces tree-shaped reachability and is needed for the $B \to M$ direction's product-of-information-states construction to be well-defined.
- *HorizonSeparation*: a sequential separation of histories by depth, also needed for $B \to M$.
- *ActionPosteriorLocal*: the conditional distribution of an action given an information state does not depend on information outside the player's recall. Sufficient for $M \to B$. Strictly weaker than perfect recall in the textbook sense.
- *PerStepPlayerRecall*: the syntactic version of ActionPosteriorLocal that is automatic for snapshot-refined information states.

The $B \to M$ direction (`kuhn_behavioral_to_mixed`) constructs the equivalent mixed profile by the independent product of the behavioral distributions across information states; the $M \to B$ direction (`kuhn_mixed_to_behavioral_semantic`) constructs the behavioral profile by extracting the information-state-conditional distributions from μ.

Textbook correspondence. Kuhn 1953 Theorem 4 (perfect-recall version); Aumann 1964 (recall hypothesis sharpening); Selten 1975 §6 (textbook treatment).

Faithfulness. Strict generalization. The textbook Kuhn's theorem requires perfect recall; the library proves both directions under strictly weaker hypotheses. The $B \to M$ direction needs no recall at all; the $M \to B$ direction needs only ActionPosteriorLocal, which holds whenever the information structure has the per-step locality property — a condition strictly weaker than full perfect recall. See chapter §6.5 for the rationale.

Variants. `Theorems/Kuhn/` contains the proof split: `ObsModel.lean` (the snapshot-refined wrapper), `KuhnModel.lean` (alternative carrier), `BehavioralToMixedCore.lean` (the $B \to M$ proof, 613 lines), `MixedToBehavioralCore.lean` (the $M \to B$ proof, 951 lines), `CorrelatedRealization.lean` (an alternative $M \to B$ proof via the correlated-equilibrium realization formula). The package also exposes generic outcome-equality schemas (`KuhnBehavioralToMixedOutcome`, `KuhnMixedToBehavioralViaOutcome`, `KuhnCompleteViaOutcome` in `Theorems/Kuhn.lean`) for language-specific Kuhn instantiations.

8 Mechanism design and auctions

8.1 Bayesian games: `Mechanism/BayesianGame.lean`

Anchor: `BayesianGame, BayesianGame.toKernelGame.`

```
structure BayesianGame (ι : Type) [Fintype ι] where
  Θ : ι → Type ; [Fintype, Nonempty]
  Act : ι → Type ; [Fintype, Nonempty]
  prior : PMF (∀ i, Θ i)
  utility : (∀ i, Θ i) → (∀ i, Act i) → ι → ℝ
abbrev Strategy (B : BayesianGame ι) (i : ι) := B.Θ i → B.Act i
noncomputable def exAnteEU (B : BayesianGame ι) (σ : B.BProfile) (who : ι) : ℝ :=
```

```
    expect B.prior (fun θ ⇒ B.utility θ (fun i ⇒ σ i (θ i)) who)
noncomputable def toKernelGame (B : BayesianGame ι) : KernelGame ι :=
    KernelGame.ofEU B.Strategy (fun σ i ⇒ B.exAnteEU σ i)
def BayesNash (B : BayesianGame ι) (σ : B.BProfile) : Prop :=
    B.toKernelGame.IsNash σ
theorem bayesNash_iff_exAnteEU (B : BayesianGame ι) (σ : B.BProfile) :
    B.BayesNash σ ↔ ∀ (who) (s'), B.exAnteEU σ who ≥ B.exAnteEU (Function.update σ who s')
who
```

Mathematical content. A Bayesian game is a tuple $(\Theta, \mathrm{Act}, \mu, u)$ with Θ_i the type space of player i, Act_i the action space, $\mu \in \mathrm{PMF}(\prod_i \Theta_i)$ the common prior, and u the type-and-action-dependent payoff. A strategy is a function $\Theta_i \to \mathrm{Act}_i$. The compilation goes through `KernelGame.ofEU`: the strategy type is preserved, the outcome type is $\mathcal{I} \to \mathbb{R}$, and the kernel is the Dirac evaluation at the ex-ante expected utility `exAnteEU`. Bayes-Nash equilibrium is exactly Nash on the compiled kernel game, characterized concretely by `bayesNash_iff_exAnteEU`.

Textbook correspondence. Harsanyi 1967 [3] type-agent representation; Osborne–Rubinstein Chapter 24.

Faithfulness. Reformulation. The textbook Bayesian Nash equilibrium quantifies over type-conditional deviations; we use the equivalent ex-ante formulation, where a deviation is a function $\Theta_i \to \mathrm{Act}_i$ replacing the player's strategy globally. The ex-post and interim variants are derived definitions. *Restriction:* finite type and action spaces.

8.2 General mechanisms and the revelation principle

Anchor: `GeneralMechanism, revelation_principle.`

```
structure GeneralMechanism (ι : Type) [Fintype ι] where
    Θ : ι → Type ; Act : ι → Type
    outcome : (∀ i, Act i) → ι → ℝ
def StrategyProfile (M : GeneralMechanism ι) := ∀ i, M.Θ i → M.Act i
def IsBNE (M : GeneralMechanism ι) (μ : PMF (∀ i, M.Θ i)) (σ : M.StrategyProfile) : Prop
noncomputable def toDirect (M : GeneralMechanism ι) (σ : M.StrategyProfile) : Mechanism ι
theorem revelation_principle (M : GeneralMechanism ι) (μ : PMF (∀ i, M.Θ i))
    (σ : M.StrategyProfile) (h : M.IsBNE μ σ) :
    (M.toDirect σ).isBIC μ
```

Mathematical content. A general mechanism has separate type spaces and action spaces; strategies map types to actions. A direct mechanism has $\mathrm{Act}_i = \Theta_i$. Bayesian incentive compatibility (BIC) of a direct mechanism is BNE under truthful reporting. The revelation principle says: for any general mechanism with a BNE σ, the direct mechanism that composes reports with σ is BIC in the library's constant-misreport sense.

Textbook correspondence. Myerson 1981 [13]; Maschler–Solan–Zamir Chapter 17.

Faithfulness. Faithful to the finite direct-mechanism construction, with one restriction: `Mechanism.isBIC` quantifies over constant misreports θ_i', whereas the stronger Bayes-Nash pred-

icate over the induced Bayesian game quantifies over arbitrary type-dependent reporting strategies. The library includes `isBIC_of_truthful_bayesNash` to relate the two levels.

8.3 Auctions: bunched

Anchor: `vickrey_truthful_dominant`.

```
noncomputable def vickreyPayoff (v : Fin n → ℝ) (bids : Fin n → ℝ) (who : Fin n) : ℝ :=
  if bids who > maxOtherBid bids who then v who - maxOtherBid bids who else 0
theorem vickrey_truthful_dominant (v : Fin n → ℝ) (who : Fin n) :
  ∀ bids b', vickreyPayoff v (Function.update bids who (v who)) who ≥
    vickreyPayoff v (Function.update bids who b') who
```

Mathematical content. In a second-price (Vickrey) auction with valuations v, the payoff is $v_i - \max_{j \neq i} b_j$ for the highest bidder (b_i) if their bid strictly exceeds the rest, and 0 otherwise. Truthful bidding $b_i = v_i$ is a weakly dominant strategy: for any opponents' bids and any alternative bid b', the payoff under truthful bidding is at least as large.

Textbook correspondence. Vickrey 1961 [22].

Faithfulness. Faithful. The proof is a case analysis on whether b' is above or below $\max_{j \neq i} b_j$, in each case verifying that the payoff under truthful bidding matches or exceeds.

Variants. `Auctions/FirstPrice.lean`, `Auctions/AllPay.lean`, `Auctions/VCG.lean`. First-price constructs the deterministic first-price payoff game and proves that no single bid is dominant. All-pay records payoff facts such as overbidding being unprofitable and rent dissipation. VCG packages the efficient-allocation setup and proves truthful reporting is dominant (Clarke 1971, Groves 1973).

8.4 Social choice

`Mechanism/SocialChoice.lean` formalizes a finite set of alternatives, a finite set of voters with strict preferences over alternatives, and structural notions: preference profiles, utilitarian aggregation, the existence of a welfare-maximizing alternative. The classical impossibility theorems (Arrow, Gibbard–Satterthwaite, Sen) are not formalized; see chapter §10.

9 Worked examples

The chapter's §6 (Worked example) traces Prisoner's Dilemma at a high level. This section gives the full Lean text and the unfolding of the relevant predicates.

9.1 Prisoner's Dilemma

```
namespace NFG
inductive PDAction where | cooperate | defect
deriving DecidableEq, Repr
def prisonersDilemma : NFGame Bool (fun _ ⇒ PDAction) where
  Outcome := ∀ _ : Bool, PDAction
  outcome := id
  utility s p :=
```

```
      match s true, s false, p with
      | defect, defect, true ⇒ 1
      | cooperate, defect, true ⇒ 0
      | defect, cooperate, true ⇒ 3
      | cooperate, cooperate, true ⇒ 2
      | ...
def pd_defect_defect :  StrategyProfile (fun _ :  Bool ⇒ PDAction) :=
   fun _ ⇒ PDAction.defect
theorem pd_defect_is_nash :  IsNashPure prisonersDilemma pd_defect_defect := by
   intro i a' ; cases i <;> cases a' <;>
      simp [prisonersDilemma, pd_defect_defect, deviate, Function.update]
```

Faithfulness check. The predicate `IsNashPure prisonersDilemma pd_defect_defect` unfolds (after `intro i a'`) to four cases, one per $(i \in \{T, F\}, a' \in \{C, D\})$ pair:

- $i = T, a' = C$: payoff at (D, D) for player T is 1, payoff at (C, D) for player T is 0. $1 \geq 0$. ✓
- $i = T, a' = D$: same profile. $1 \geq 1$. ✓
- $i = F, a' = C$: payoff at (D, D) for F is 1, payoff at (D, C) for F is 0. $1 \geq 0$. ✓
- $i = F, a' = D$: same profile. $1 \geq 1$. ✓

Each inequality is the textbook Nash check. The Lean proof closes all four with one `simp`.

The companion theorem `pd_coop_not_nash : Âň IsNashPure prisonersDilemma pd_coop_coop` exhibits a profitable deviation: player T deviates from C to D and gains payoff $3 - 2 = 1$. The Lean proof extracts the T, D instance of IsNash, simplifies, and closes with `linarith`.

9.2 Matching Pennies

```
inductive MPAction where | heads | tails
def matchingPennies :  NFGGame Bool (fun _ ⇒ MPAction) where
   Outcome := ∀ _ :  Bool, MPAction
   outcome := id
   utility := - (H,H) = (1,-1), (H,T) = (-1,1), ...
```

Faithfulness check. There is no pure Nash equilibrium: at (H, H) player F deviates to T and gains $+2$; at (H, T) player T deviates to T and gains $+2$; the four pure profiles are similarly checked. The textbook mixed equilibrium is the symmetric uniform profile $\sigma_T^* = \sigma_F^* = \frac{1}{2}\delta_H + \frac{1}{2}\delta_T$, at which both players' EU is 0. The Lean example records the no-pure-Nash result; it does not package the mixed-equilibrium calculation as a theorem in `Languages/NFG/Examples.lean`.

9.3 A two-step centipede in EFG

A two-step centipede: player 0 chooses Take or Pass; if Pass, player 1 chooses Take or Pass; payoffs $\mathrm{TT}_0 = (1, 0)$, $\mathrm{PT} = (0, 2)$, $\mathrm{PP} = (3, 1)$. The textbook backward-induction solution is (T, T): at the second node, player 1 strictly prefers T $(2 > 1)$, so they play T; at the first node, player 0 anticipates this and prefers T $(1 > 0)$. The library's packaged extensive-form example is the entry-deterrence game in `Languages/EFG/Refinements.lean`, where it proves:

- `entrySPE_isSPE`: the backward-induction profile is SPE.

- `entryNash_isNash`: another profile is Nash.
- `entryNash_not_spe`: that Nash profile is not SPE.

The two-step centipede calculation is illustrative prose, not a separate Lean theorem.

9.4 A small Bayesian game

A two-player coordination game with binary types: each player has type $\theta_i \in \{0, 1\}$ drawn independently uniform; actions are $a_i \in \{L, R\}$; payoffs are $u_i(\theta, a) = [\![a_T = a_F]\!] \cdot [\![a_T = \theta_T]\!]$. The Bayes-Nash equilibrium has each player play $a_i = \theta_i$; the Lean proof unfolds `bayesNash_iff_exAnteEU` and verifies the four-deviation inequality directly under the uniform prior.

10 Reading order

Three suggested traversals.

(a) "I want to use the library on a finite NFG." §2 (probability primitives) → §3.1, §3.2, §3.3 (semantic core) → §4.3 (Nash) → §5.1 (NFG) → §7.1 (existence). §9.1 is the canonical worked example.

(b) "I want to add a new language." §3 (semantic core) → §5.1 or §5.2 as a template → §6.1 (Language/Reduction abstraction) to register the language for bridges. The worked subclass files inside `Languages/NFG/` (CongestionGame, Bargaining, Stackelberg) are useful as additional templates.

(c) "I want to follow the Kuhn theorem proof end-to-end." §2 → §3.1, §3.2 → §5.2 (EFG behavioral profiles) → §7.5 (the four hypotheses, then the two anchor theorems). The proof split is in `BehavioralToMixedCore.lean` and `MixedToBehavioralCore.lean` in `Theorems/Kuhn/`; the `CorrelatedRealization.lean` alternative goes via the mixed-Nash-induced correlated distribution.

References

[1] Robert J. Aumann. Subjectivity and correlation in randomized strategies. *Journal of Mathematical Economics*, 1(1):67–96, 1974.

[2] Robert J. Aumann. Agreeing to disagree. *Annals of Statistics*, 4(6):1236–1239, 1976.

[3] John C. Harsanyi. Games with incomplete information played by "Bayesian" players, parts I–III. *Management Science*, 14:159–182, 320–334, 486–502, 1967.

[4] Sergiu Hart and Andreu Mas-Colell. A simple adaptive procedure leading to correlated equilibrium. *Econometrica*, 68(5):1127–1150, 2000.

[5] Benjamin Heymann, Michel De Lara, and Jean-Philippe Chancelier. Kuhn's equivalence theorem for games in intrinsic form, 2020.

[6] Daphne Koller and Brian Milch. Multi-agent influence diagrams for representing and solving games. *Games and Economic Behavior*, 45:181–221, 2003.

[7] Elias Koutsoupias and Christos Papadimitriou. Worst-case equilibria. In *STACS*, 1999.

[8] Vojtěch Kovařík, Martin Schmid, Neil Burch, Michael Bowling, and Viliam Lisý. Rethinking formal models of partially observable multi-agent decision making. *Artificial Intelligence*, 303, 2022.

[9] Harold W. Kuhn. Extensive games and the problem of information. In H. W. Kuhn and A. W. Tucker, editors, *Contributions to the Theory of Games, Vol. II*, pages 193–216. Princeton University Press, 1953.

[10] Michael Maschler, Eilon Solan, and Shmuel Zamir. *Game Theory*. Cambridge University Press, 2013.

[11] John Maynard Smith and George R. Price. The logic of animal conflict. *Nature*, 246:15–18, 1973.

[12] Dov Monderer and Lloyd S. Shapley. Potential games. *Games and Economic Behavior*, 14: 124–143, 1996.

[13] Roger B. Myerson. Optimal auction design. *Mathematics of Operations Research*, 6:58–73, 1981.

[14] Roger B. Myerson. *Game Theory: Analysis of Conflict*. Harvard University Press, 1991.

[15] John F. Nash. Non-cooperative games. *Annals of Mathematics*, 54(2):286–295, 1951.

[16] Martin J. Osborne and Ariel Rubinstein. *A Course in Game Theory*. MIT Press, 1994.

[17] Tim Roughgarden. *Selfish Routing and the Price of Anarchy*. MIT Press, 2005.

[18] Reinhard Selten. Spieltheoretische Behandlung eines Oligopolmodells mit Nachfrageträgheit. *Zeitschrift für die gesamte Staatswissenschaft*, 121:301–324, 667–689, 1965.

[19] Reinhard Selten. Reexamination of the perfectness concept for equilibrium points in extensive games. *International Journal of Game Theory*, 4:25–55, 1975.

[20] Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39: 1095–1100, 1953.

[21] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.

[22] William Vickrey. Counterspeculation, auctions, and competitive sealed tenders. *Journal of Finance*, 16:8–37, 1961.

[23] John von Neumann. Zur Theorie der Gesellschaftsspiele. *Mathematische Annalen*, 100:295–320, 1928.

[24] John von Neumann and Oskar Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press, 1944.